Information Retrieval and Extraction : Lecture IV

Roshni Chakraborty

Dept. Of Information Technology

ABV-IIITM Gwalior

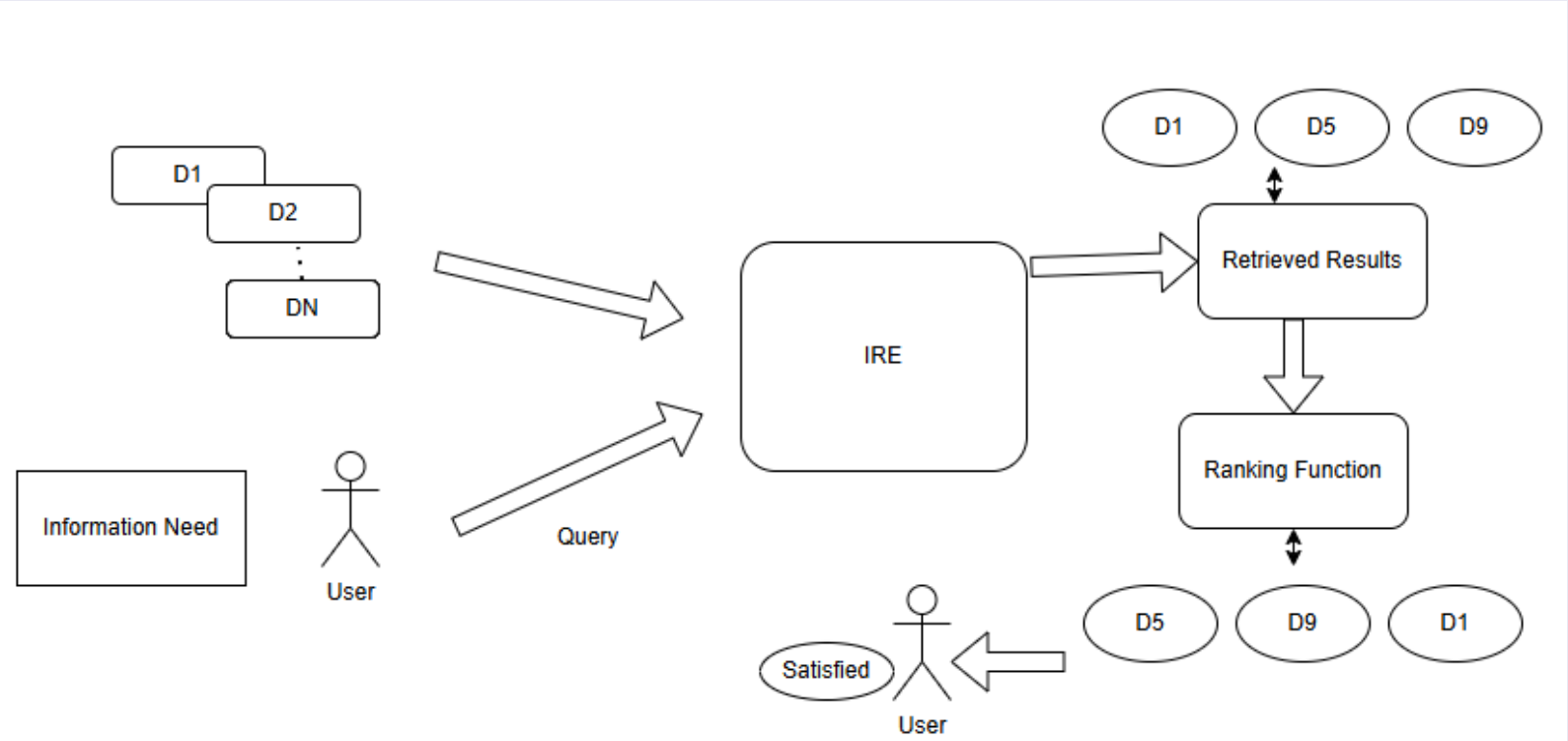
February 11, 2026

Outline I

1. Overview
2. Processing Queries
   - Postings Lists
   - Spelling correction
   - K-gram Indexes
3. Evaluation Metrics

Overview

Recap



Processing Queries

Intersection of Posting Lists

```
INTERSECT(p1, p2)
  answer ← ⟨ ⟩
  while p1 ≠ NIL and p2 ≠ NIL
  do if docID(p1) = docID(p2)
        then ADD(answer, docID(p1))
             p1 ← next(p1)
             p2 ← next(p2)
        else if docID(p1) < docID(p2)
                then p1 ← next(p1)
                else p2 ← next(p2)
  return answer
```

Processing Queries

Intersection of Posting Lists

- Efficiently intersect postings lists
- Merging the lists with a logical **AND** operation
- Query Optimization
  - process of selecting how to organize the work
  - reduce amount of work done in answering a query
  - examples : Brutus AND Caesar AND Calpurnia
    - start by intersecting the two smallest postings lists
    - ordering decision based on in-memory data

Processing Queries

Spelling corrections

- User wish to retrieve documents containing the term carrot
  - when the user types the query *carot*
- Solutions
  - based on edit distance
  - based on k-gram overlap
- For evert alternative correct spellings for a mis-spelled query
  - choose the "nearest" one

Things to Remember

- carot always retrieve documents containing carot as well as any "spell-corrected"
  - carrot and tarot
- when query **carot** does not exist
- when query **carot** exists but returns fewer documents than preset
  - then, it finds the spell corrected version and asks user

Processing Queries

Implementing Spelling Correction

- When two correctly spelled queries are tied
  - choose the one which is more **probable**
  - **grunt** and **grant** for **grnt**
- More **probable**
  - consider the number of occurrences of the term in the collection
  - use the correction that is most common among queries typed in by other users
  - cricket

Processing Queries

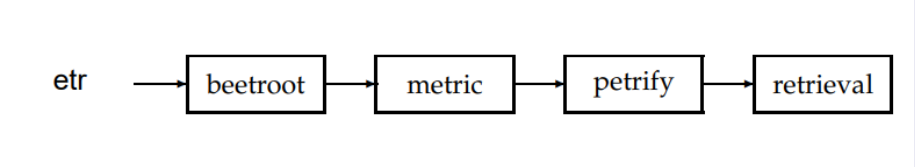
Levenshtein Distance

- Given two character strings s1 and s2
  - edit distance between them is the minimum number of edit operations required to transform s1 into s2
- Edit operations
  - insert a character into a string
  - delete a character from a string
  - replace a character of a string by another character
- edit distance can be generalized to allowing different weights for different kinds of edit operations
  - consider key placement
- spelling correction problem demands more than computing edit distance
  - given a set of strings and the query string
  - find the string with least of edit distance

Processing Queries

k-gram indexes

- A k-gram is a sequence of k characters
- *cas*, *ast* and *stl* are all 3-grams occurring in the term castle
  - use a special character to denote the beginning or end
  - *ca*, *cas*, *ast*, *stl*, *tle*, *le*
- etr occurs in ?
- In a k-gram index,
  - the dictionary contains all k-grams
  - that occur in any term in the vocabulary



Processing Queries

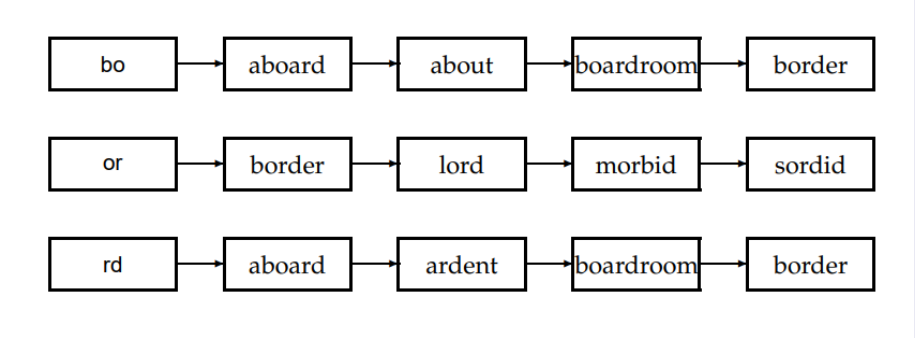
k-gram indexes

- How does such an index help us with wildcard queries?
  - Consider the query *re*ve*
  - Find any document that starts with *re* and ends with *ve*
- Methodology
  - run the Boolean query $*re* AND *ve*$
  - Find matching terms
  - Each matching term mapped to documents
- If its a combination, such as, re*d AND fe*ri
  - seek all documents
  - contain any term matching re*d AND
  - any term matching fe*r

Processing Queries

k-gram indexes

- How to filter retrieved tokens to ensure least calculation?
  - For three bigrams, in the query **bord**
  - Need to enumerate all possible combinations



Processing Queries

k-gram indexes : Jaccard coefficient

- Given A is the set of k-grams in the query q,
- and, B is the set of k-grams in a vocabulary term, t
- Calculate *Jaccard coefficient*
  - $JC(A, B) = \frac{A \cap B}{A \cup B}$
- If $JC(A, B) > \theta$, we add t to the output

Processing Queries

Context sensitive spelling correction

- Isolated Term Correction
  - we attempt to correct a single query term at a time
- What about the query *flew form Heathrow*
  - enumerate corrections of each of the three query terms
  - ex : *fled form Heathrow* and *flew fore Heathrow*
  - retain only the most frequent combinations
    - in the collection or in the query logs
    - based on *previous query* history of other users

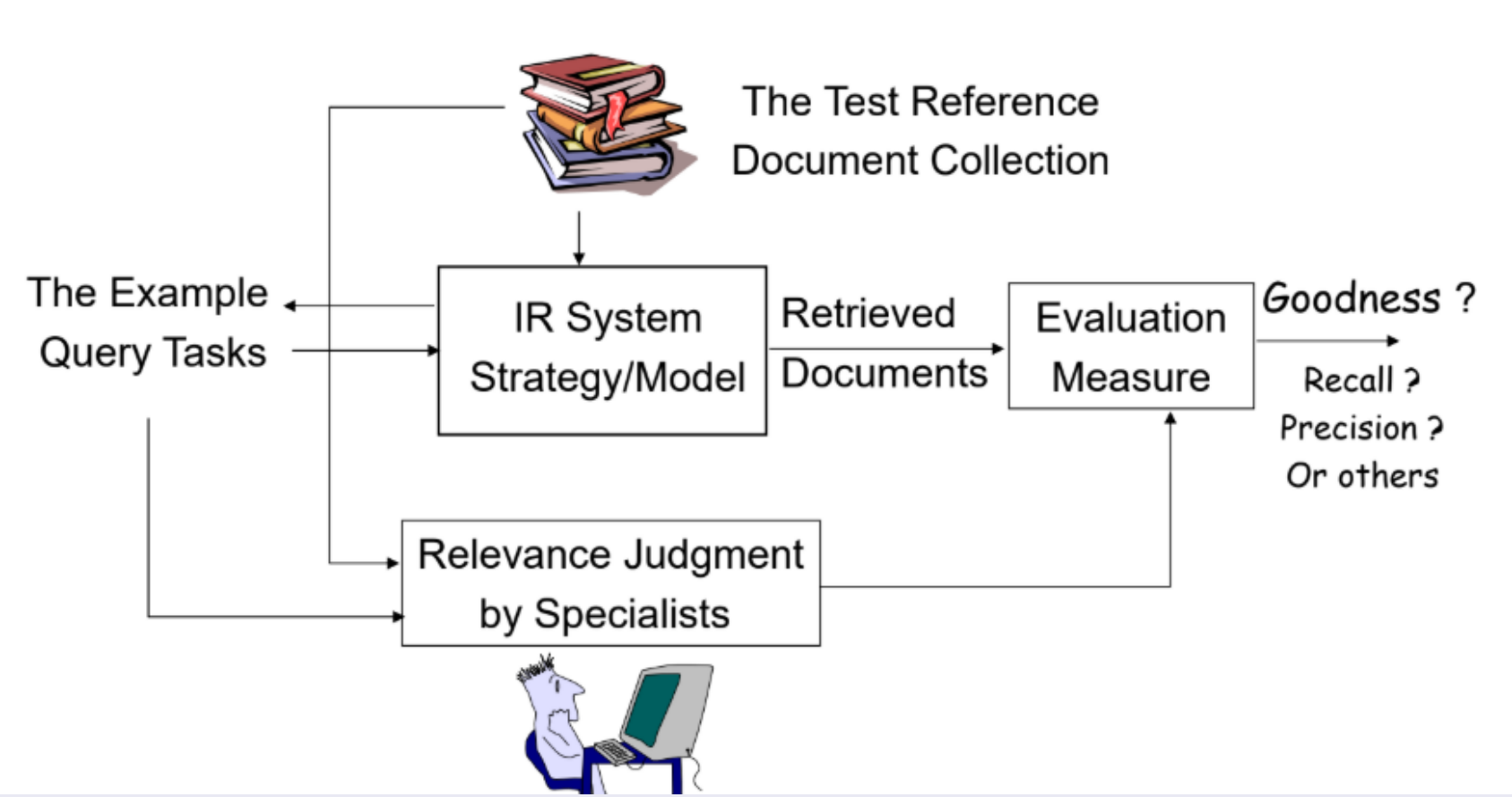
Evaluation Metrics

Recap

- Performance evaluation
  - Data Retrieval System (shorter response time, less space used, trade-off)
    - Not relevant for IRE currently
  - Retrieval evaluation
    - Relevance of retrieved documents is important
    - user's query is inherently vague
    - can lead to not exact answers
    - needs to be ranked according to their relevance to the query
- Comparison between the results generated automatically with *human generated responses*
  - Require availability of human generated responses

Evaluation Metrics

Recap



References

- Introduction to Information Retrieval by Christopher Manning (ch-2 and ch-3)
  - Ch2 : The term vocabulary and postings lists
  - Ch 3 : Dictionaries and tolerant retrieval

Thank you!